

```

In [11]: # ----- 设置中文字体 -----
# 指定支持中文的字体 (SimHei=黑体, Microsoft YaHei=微软雅黑), 避免标题变成方框
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'PingFang SC']
# 让负号正常显示 (不使用 unicode 减号)
plt.rcParams['axes.unicode_minus'] = False

# ----- 学生数据 -----
# 每个学生是一行字典, 包含姓名、性别(male/female)、年龄、班级、语数英三科成绩
students = [
    {'name': '张三', 'gender': 'male', 'age': 18, 'class': 'class1', 'chinese': 85},
    {'name': '李四', 'gender': 'male', 'age': 16, 'class': 'class2', 'chinese': 78},
    {'name': '王五', 'gender': 'female', 'age': 15, 'class': 'class3', 'chinese': 92},
    {'name': '赵六', 'gender': 'female', 'age': 17, 'class': 'class4', 'chinese': 88},
    {'name': '钱七', 'gender': 'male', 'age': 18, 'class': 'class1', 'chinese': 75},
    {'name': '孙八', 'gender': 'male', 'age': 19, 'class': 'class2', 'chinese': 82},
    {'name': '周九', 'gender': 'female', 'age': 18, 'class': 'class3', 'chinese': 90},
    {'name': '吴十', 'gender': 'female', 'age': 17, 'class': 'class4', 'chinese': 80}
]

# =====
# 任务1: 统计男女学生人数 (柱状图)
# plt.bar() — 绘制分类对比柱状图
# =====

# 列表推导式: 分别统计 male 和 female 的人数
male_cnt = sum(1 for s in students if s["gender"] == "male")
female_cnt = sum(1 for s in students if s["gender"] == "female")

# 创建一个 8x5 英寸的画布和坐标轴
fig, ax = plt.subplots(figsize=(8, 5))

# bar(x, height) — 柱状图, width 控制柱宽, edgecolor 给边框上色
bars = ax.bar(["男", "女"], [male_cnt, female_cnt],
               color=["steelblue", "coral"], width=0.4, edgecolor="black")

# bar_label() — 在柱状图顶部自动显示数值 (比手动 text 更方便)
ax.bar_label(bars, fontsize=13)

# 设置标题、y 轴标签和网格线
ax.set_title("男女人数统计", fontsize=15)
ax.set_ylabel("人数")
ax.grid(axis="y", alpha=0.3) # 只在 y 方向显示网格, 半透明

plt.tight_layout() # 自动调整子图间距, 防止标签被裁切
plt.show()

# =====
# 任务2: 语文成绩分段统计 (直方图)
# plt.hist() — 展示数据分布情况
# =====

# 提取所有学生的语文成绩
chinese_scores = [s["chinese"] for s in students]

fig, ax = plt.subplots(figsize=(10, 5))

# hist(data, bins) — 直方图, bins=5 表示分成 5 个等距区间
# edgecolor 给每个柱子加边框, alpha 控制透明度

```

```

counts, bins, patches = ax.hist(chinese_scores, bins=5, edgecolor="black",
                                color="lightgreen", alpha=0.8)

ax.set_title("语文成绩分段统计", fontsize=15)
ax.set_xlabel("分数段")
ax.set_ylabel("人数")

# 在每根柱子的顶部标注具体人数
# counts 是每个区间的频数, bins 是区间边界
for count, bin_edge in zip(counts, bins):
    if count > 0: # 只标注有数据的柱子
        ax.text(bin_edge + (bins[1]-bins[0])/2, # x 坐标: 区间中点
                 count + 0.1, # y 坐标: 柱顶稍微往上
                 int(count), ha="center", fontsize=12)

ax.set_xticks(bins) # 让 x 轴刻度对齐区间边界
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# 任务3: 语文与英语成绩相关性分析 (散点图)
# plt.scatter() — 探索两个变量的关联性
# =====

# 提取语、英成绩和学生姓名
chinese = [s["chinese"] for s in students]
english = [s["english"] for s in students]
names = [s["name"] for s in students]

# corrcoeff 计算皮尔逊相关系数矩阵, 取 [0,1] 即 r 值
corr = np.corrcoef(chinese, english)[0, 1]

fig, ax = plt.subplots(figsize=(10, 6))

# scatter(x, y) — 散点图, s 控制点大小, c 控制颜色, alpha 透明度
# zorder 控制图层顺序, 越大越靠上
ax.scatter(chinese, english, s=80, c="purple", alpha=0.7, zorder=3)

# 用 annotate 在每个点旁边标注学生姓名
# xytext=(5,5) 表示文本偏移 (5px, 5px)
for i, name in enumerate(names):
    ax.annotate(name, (chinese[i], english[i]),
                textcoords="offset points", xytext=(5, 5), fontsize=9)

# polyfit 做一次线性拟合 (1 次多项式), poly1d 构建多项式函数
z = np.polyfit(chinese, english, 1)
p = np.poly1d(z)

# 生成拟合线的 x 范围, 并计算对应的 y
x_fit = np.linspace(min(chinese)-2, max(chinese)+2, 50)
ax.plot(x_fit, p(x_fit), "r--", lw=2, alpha=0.7,
        label=f"拟合线 (r={corr:.3f})" # r 值显示在图例中

ax.set_title("语文与英语成绩相关性分析", fontsize=15)
ax.set_xlabel("语文成绩")
ax.set_ylabel("英语成绩")
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()

```

```
plt.show()

# =====
# 任务4: 年龄与数学平均成绩的关系 (折线图)
# plt.plot() — 反映趋势变化
# =====

from collections import defaultdict

# defaultdict(list) 自动为每个年龄创建一个空列表
# 把相同年龄的数学成绩收集到一起
age_math = defaultdict(list)
for s in students:
    age_math[s["age"]].append(s["math"])

# 年龄排序, 计算每个年龄的数学平均分
ages = sorted(age_math.keys())
avg_math = [np.mean(age_math[a]) for a in ages]

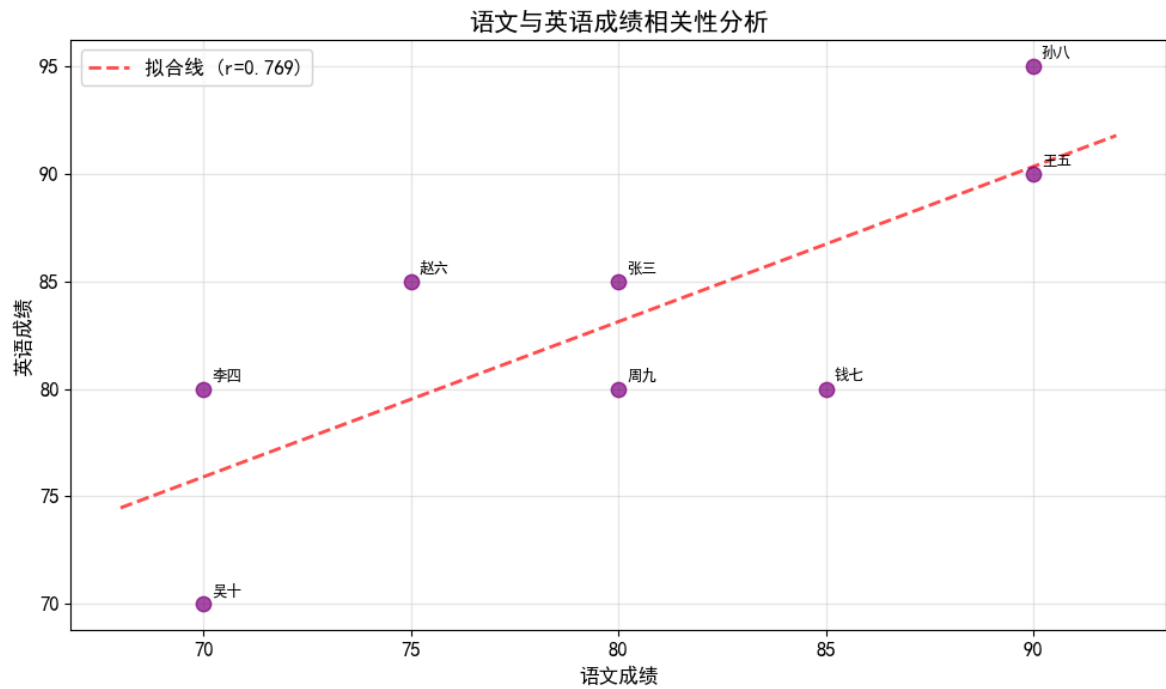
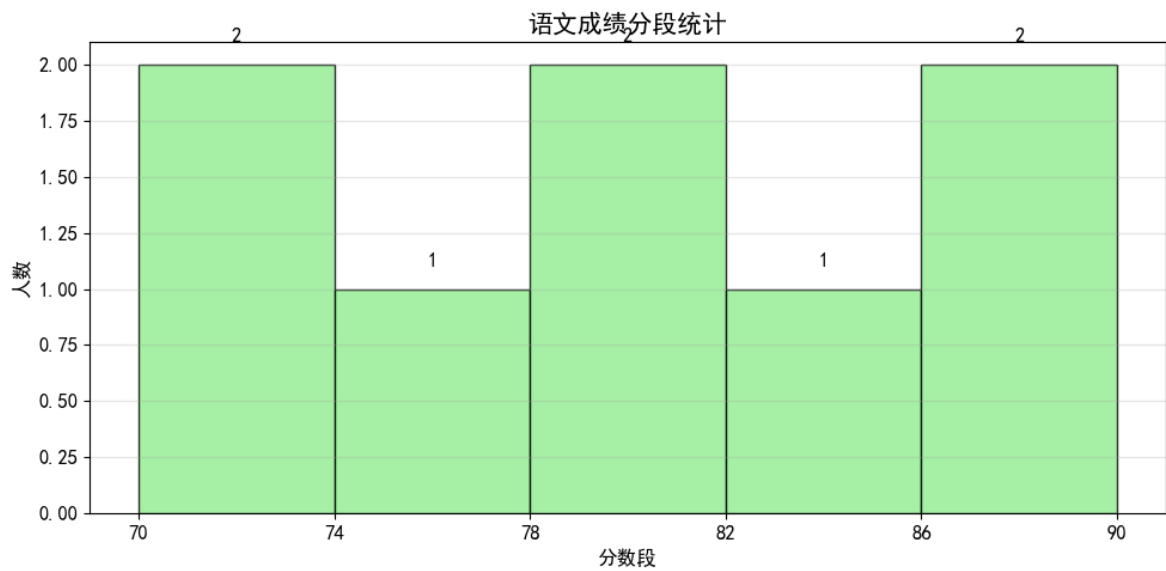
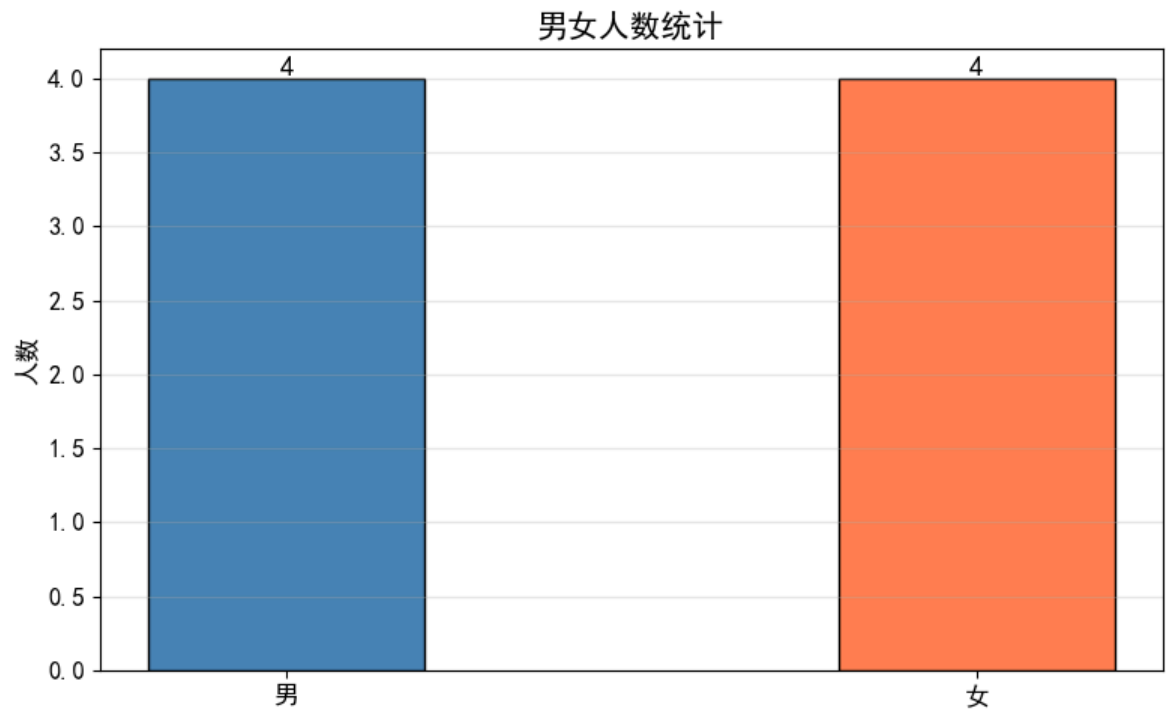
fig, ax = plt.subplots(figsize=(10, 5))

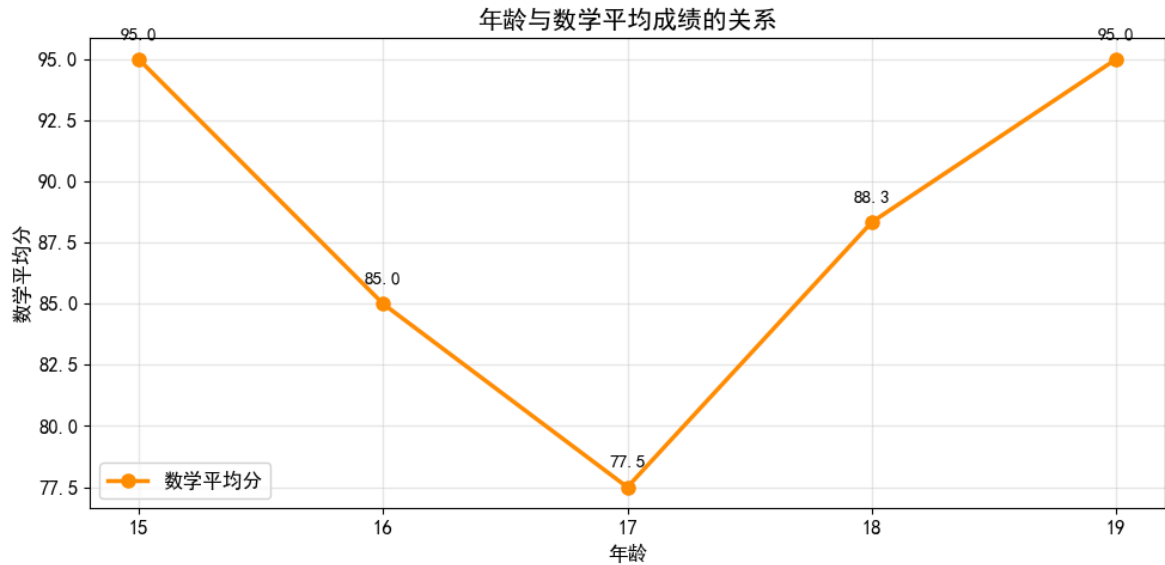
# plot(x, y, "o-") — 折线图 + 圆点标记
# lw 线宽, ms 标记点大小
ax.plot(ages, avg_math, "o-", color="darkorange", lw=2.5, ms=8,
        label="数学平均分")

# 在每个数据点上方标注具体数值
for a, m in zip(ages, avg_math):
    ax.text(a, m + 0.8, f"{m:.1f}", ha="center", fontsize=11)

ax.set_title("年龄与数学平均成绩的关系", fontsize=15)
ax.set_xlabel("年龄")
ax.set_ylabel("数学平均分")
ax.set_xticks(ages) # x 轴只显示有数据的年龄
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print("✅ 成绩可视化分析完成!")
```





✅ 成绩可视化分析完成！

```
In [15]: import numpy as np
import matplotlib.pyplot as plt

# =====
# 数据：矩形用 [x1, y1, x2, y2] 表示（左下角，右上角）
# — 采用标准笛卡尔坐标系，y 轴向上为正
# =====
box = np.array([ 2,  2, 20, 15]) # 候选框 (candidate)
boxes = np.array([15, 12, 25, 21]) # 真实框 (ground truth)

# =====
# 1. 画出两个矩形
# =====
fig, ax = plt.subplots(figsize=(6, 6))

# box 矩形 (蓝色)
rect1 = plt.Rectangle((box[0], box[1]), box[2]-box[0], box[3]-box[1],
                      fill=False, edgecolor='blue', lw=3, label='Candidate')
ax.add_patch(rect1)

# boxes 矩形 (红色)
rect2 = plt.Rectangle((boxes[0], boxes[1]), boxes[2]-boxes[0], boxes[3]-boxes[1],
                      fill=False, edgecolor='red', lw=3, label='Ground Truth')
ax.add_patch(rect2)

# 标注四个角坐标
ax.text(box[0], box[1], f'({box[0]},{box[1]})', fontsize=9, ha='right', va='top')
ax.text(box[2], box[3], f'({box[2]},{box[3]})', fontsize=9, ha='left', va='bottom')
ax.text(boxes[0], boxes[1], f'({boxes[0]},{boxes[1]})', fontsize=9, ha='right', va='top')
ax.text(boxes[2], boxes[3], f'({boxes[2]},{boxes[3]})', fontsize=9, ha='left', va='bottom')

ax.set_xlim(0, 30); ax.set_ylim(0, 25)
ax.set_aspect('equal') # 保持纵横比一致
ax.set_xlabel('x'); ax.set_ylabel('y')
ax.legend(); ax.grid(alpha=0.3)
ax.set_title('Two Rectangles (Cartesian Coordinates)')
plt.show()

# =====
# 2. 计算各自面积
# =====
```

```

area_box    = (box[2] - box[0]) * (box[3] - box[1])
area_boxes  = (boxes[2] - boxes[0]) * (boxes[3] - boxes[1])

print(f"Candidate area: {area_box}")
print(f"Ground Truth area: {area_boxes}")

# =====
# 3. 计算交集面积
# 核心: 交集左下角 = max(各自左下角)
#        交集右上角 = min(各自右上角)
# =====
x1 = np.maximum(box[0], boxes[0]) # max(2, 15) = 15
y1 = np.maximum(box[1], boxes[1]) # max(2, 12) = 12
x2 = np.minimum(box[2], boxes[2]) # min(20, 25) = 20
y2 = np.minimum(box[3], boxes[3]) # min(15, 21) = 15

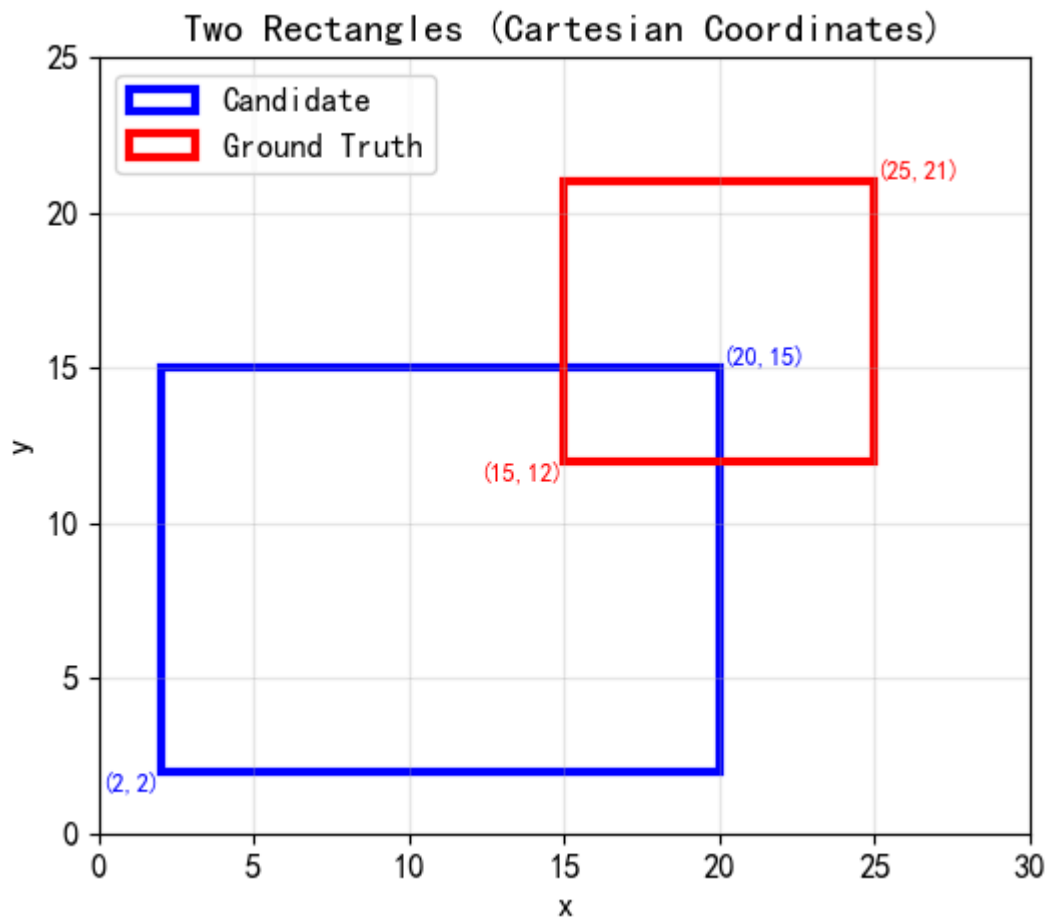
inter_w = np.maximum(0, x2 - x1) # 20 - 15 = 5
inter_h = np.maximum(0, y2 - y1) # 15 - 12 = 3
inter_area = inter_w * inter_h   # 5 * 3 = 15

print(f"\nIntersection: x1={x1}, y1={y1}, x2={x2}, y2={y2}")
print(f"Intersection width={inter_w}, height={inter_h}")
print(f"Intersection area = {inter_area}")

# =====
# 4. np.maximum / np.minimum 示范
# =====
a = np.array([1, 2, 3, 4])
b = np.array([0, 3, 2, 1])

print(f"\n--- np.maximum / np.minimum 演示 ---")
print(f"a = {a}")
print(f"b = {b}")
print(f"np.maximum(a, b) = {np.maximum(a, b)}") # 逐个取大: [1, 3, 3, 4]
print(f"np.minimum(a, b) = {np.minimum(a, b)}") # 逐个取小: [0, 2, 2, 1]

```



Candidate area: 234

Ground Truth area: 90

Intersection: x1=15, y1=12, x2=20, y2=15

Intersection width=5, height=3

Intersection area = 15

--- np.maximum / np.minimum 演示 ---

a = [1 2 3 4]

b = [0 3 2 1]

np.maximum(a, b) = [1 3 3 4]

np.minimum(a, b) = [0 2 2 1]

```
In [16]: # =====
# 5. 求出前3个最相似特征的姓名
# 数据: file/target_feat.txt (目标特征)
#       file/feat.txt (特征库, 格式: id|姓名|[特征向量])
# 方法: 余弦相似度 (cosine similarity)
# =====
import numpy as np

# ----- 读取目标特征 -----
with open('file/target_feat.txt', 'r', encoding='utf-8') as f:
    target_str = f.read().strip()
    target_feat = np.array(eval(target_str)) # 字符串转数组

# ----- 读取特征库 -----
names = [] # 存储姓名
features = [] # 存储特征向量
with open('file/feat.txt', 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
```

```

        if not line:
            continue
        # 格式: id|姓名|[特征向量]
        parts = line.split('|')
        idx = parts[0]
        name = parts[1]
        feat_str = '|'.join(parts[2:]) # 特征向量部分
        feat = np.array(eval(feat_str))
        names.append(name)
        features.append(feat)

features = np.array(features)

print(f"目标特征维度: {target_feat.shape}")
print(f"特征库总数: {len(names)}")

# ----- 计算余弦相似度 -----
# 余弦相似度 = (A·B) / (||A|| * ||B||)
target_norm = np.linalg.norm(target_feat)
features_norm = np.linalg.norm(features, axis=1)
dot_products = np.dot(features, target_feat)
similarities = dot_products / (features_norm * target_norm + 1e-8)

# ----- 找出目标自身 (相似度 ≈ 1.0) -----
self_idx = np.argmax(similarities)
print(f"\n目标自身: id={self_idx}, name='{names[self_idx]}', similarity={similarities[self_idx]}")

# ----- 排除自身, 取前3 -----
top_k = 3
similarities_copy = similarities.copy()
similarities_copy[self_idx] = -1 # 自身置为最低
top_indices = np.argsort(similarities_copy)[::-1][:top_k]

print(f"\nTOP 前{top_k}个最相似特征 (排除自身):")
print(f"{'排名':<6}{'姓名':<10}{'相似度':<10}{'原索引'}")
print("-" * 36)
for rank, idx in enumerate(top_indices, 1):
    print(f"{'rank':<6}{'names[idx]:<10}{similarities[idx]:.4f} (id={idx})")

```

目标特征维度: (128,)

特征库总数: 11

目标自身: id=10, name='XY', similarity=1.0000

前3个最相似特征 (排除自身):

排名	姓名	相似度	原索引
1	XY	0.7813	(id=9)
2	XY	0.7303	(id=8)
3	d1	0.1941	(id=1)